

Tehtävänanto, lopputyö, osa 3 (10 pistettä):

Havainnollista automaattisen koneoppimisen (AutoML) -asioita (TPOT) Jupiter Notebookilla.

Käytä AutoML-havainnollistamiseen Harjoituksen 7 datajoukkoja, jollet löydä kahta datasettiä, joista toinen on leimattu (labelled) ja toinen ennakointia varten (unlabelled). 5 op suorittavat palauttavat Jupiter Notebook -tiedoston, jossa on mukana tekstiä, joista ilmenee perustelut asioille, joita on tehty datan muokkaamiseksi TPOT:lle sopivaksi ja potentiaalisten datajoukon muokkaamiseksi (vinkkinä on, että voit käyttää Exceliä tai RapidMinerin Turbo Prep -ominaisuutta datajoukkojen muokkaamiseen), jotta TPOT:n luomaa mallia voidaan käyttää sekä tietenkin selkeästi potentiaaliset ostajat.

Jupiter Notebook tiedoston sisältö:



```
# Asennetaan tarvittavat paketit
!pip install tpot
```



```
# Ladataan tarvittavat kirjastot
from tpot import TPOTClassifier
from sklearn.model_selection import train_test_split
import pandas as pd
import numpy as np
```

```

# Ladataan data

##bike_data seka bike_data_potentialiset on muokattu seuraavalla tavalla:
#sarakeet nimetty samannimisiksi (esim. yearly income ja income nimetty samannimisiksi)
#puuttuvat arvot on korvattu keskiarvoisilla arvoilla
#bike_data_potentialisista on poistettu kaikki ylimääräiset sarakeet (kuten nimi, sposti, osoite)
#eli sarakeita on molemmissa tauluissa yhtä monta.
#kaikki sarakeet on laitettu samaan järjestykseen

#Bike_data_potentialiset:ssa target arvoilla on annettu dummy-arvo -999
#tämä, koska -999 on ennalta-arvattavampi arvo kuin null tai tyhjä tspotin
#käytön suhteen

# Alla oleva koodi ei toimi
#bike = pd.read_csv('bike_data.csv')
#bike.head()

# Yllä oleva koodi korvattu alla olevalla
from google.colab import drive
drive.mount('/content/drive')

import os
bike_file_path = '/content/drive/My Drive/Bike_data.csv'

bike = pd.read_csv(bike_file_path)
bike.head()

```

	ID	Income	Children	Cars	Age	target	Marital Status = Married	Marital Status = Single	Gender = Female	Gender = Male	...
0	-1.396956	-0.519278	-0.558393	-1.291005	-0.192891	No	0.924355	-0.924355	1.017656	-1.017656	...
1	0.774406	-0.841012	0.671548	-0.401883	-0.104813	No	0.924355	-0.924355	-0.981668	0.981668	...
2	-1.082594	0.767657	1.901490	0.487239	1.392518	No	0.924355	-0.924355	-0.981668	0.981668	...
3	0.825647	0.445923	-1.173364	-0.401883	-0.280970	Yes	-1.080754	1.080754	-0.981668	0.981668	...
4	1.053050	-0.841012	-1.173364	-1.291005	-0.721361	Yes	-1.080754	1.080754	-0.981668	0.981668	...

5 rows × 28 columns

```

#Katsotaan miten monta rivia ja saraketta datassa on
bike.shape

```

```

(1000, 28)

```

```

#Katsotaan datan datatyyppi (joka nyt on pandas)
type(bike)

```

```

pandas.core.frame.DataFrame
def __init__(data=None, index: Axes | None=None, columns: Axes | None=None, dtype: Dtype |
None=None, copy: bool | None=None) -> None

```

Two-dimensional, size-mutable, potentially heterogeneous tabular data.

Data structure also contains labeled axes (rows and columns). Arithmetic operations align on both row and column labels. Can be thought of as a dict-like container for Series objects. The primary pandas data structure.



```
#muuteetaan target kentät yes ja no arvoiksi 1 ja 0
bike['target']=bike['target'].map({'No':0,'Yes':1})
bike.head()
```



	ID	Income	Children	Cars	Age	target	Marital Status = Married	Marital Status = Single	Gender = Female	Gender = Male	...
0	-1.396956	-0.519278	-0.558393	-1.291005	-0.192891	0	0.924355	-0.924355	1.017656	-1.017656	...
1	0.774406	-0.841012	0.671548	-0.401883	-0.104813	0	0.924355	-0.924355	-0.981668	0.981668	...
2	-1.082594	0.767657	1.901490	0.487239	1.392518	0	0.924355	-0.924355	-0.981668	0.981668	...
3	0.825647	0.445923	-1.173364	-0.401883	-0.280970	1	-1.080754	1.080754	-0.981668	0.981668	...
4	1.053050	-0.841012	-1.173364	-1.291005	-0.721361	1	-1.080754	1.080754	-0.981668	0.981668	...

5 rows × 28 columns

```
[ ] #Luodaan uusi bike data, jossa ei ole kenttiä ID ja target
bike_new = bike.drop(['ID','target'], axis=1)
```

```
[ ] #Katsotaan bike_new:n datatyyppi, joka nyt on pandas
type(bike_new)
```



```
pandas.core.frame.DataFrame
def __init__(data=None, index: Axes | None=None, columns: Axes | None=None, dtype: Dtype |
None=None, copy: bool | None=None) -> None
```

Two-dimensional, size-mutable, potentially heterogeneous tabular data.

Data structure also contains labeled axes (rows and columns). Arithmetic operations align on both row and column labels. Can be thought of as a dict-like container for Series objects. The primary pandas data structure.

```
[ ] #muunnetaan .values komennolla bike_new tiettyyppiin numpy
#uusi nimi bike_newest
#data pitää muuntaa numpyksi, tai tpotin koulutus ei onnistu
bike_newest = bike_new.values
```

```
[ ] #katsotaan mitä bike_newest pitää sisällään
bike_newest[:,:]
```

```
⇒ array([[ -0.51927812, -0.55839346, -1.29100539, ..., -0.35317776,
          -0.48722288, -1.01562188],
         [ -0.84101178,  0.67154809, -0.40188322, ..., -0.35317776,
          -0.48722288, -1.01562188],
         [  0.76765651,  1.90148964,  0.48723895, ..., -0.35317776,
          -0.48722288, -1.01562188],
         ...,
         [  0.12418919,  0.05657731, -1.29100539, ..., -0.35317776,
          -0.48722288,  0.98363379],
         [  1.41112382,  0.67154809,  1.37636112, ..., -0.35317776,
          -0.48722288,  0.98363379],
         [  0.12418919,  0.67154809,  0.48723895, ...,  2.8286039 ,
          -0.48722288,  0.98363379]])
```

```
[ ] #tarkistetaan, että tietotyyppi on numpy
type(bike_newest)
```

```
⇒ numpy.ndarray
```

```
[ ] #Luodaan target datajoukko, jonka perusteella
#tekoälyn koulutus katsoo onko rivin arvo 0 vai 1
```

```
bike_target = bike['target'].values
```

```
#katsotaan bike_target sisältö
bike_target[:]
```

```

array([0, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0,
       1, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 1,
       1, 1, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 1, 0, 1, 0,
       1, 1, 1, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 1, 1, 0, 1, 0, 1, 1, 1, 0,
       0, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1, 0, 0, 1, 0, 1, 1, 1, 1,
       1, 0, 0, 1, 1, 1, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 1, 0,
       1, 1, 0, 0, 1, 0, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 0, 1, 0, 0, 0,
       0, 1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 1,
       1, 0, 1, 1, 0, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 0, 1, 0, 1,
       1, 1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 0, 0, 0, 1,
       1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 1, 1, 1, 0, 1, 0, 0,
       1, 0, 1, 1, 1, 1, 0, 1, 1, 0, 0, 0, 1, 0, 1, 1, 1, 1, 0, 1, 0, 0,
       1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1,
       0, 1, 0, 0, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1, 0,
       0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0,
       1, 0, 1, 0, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 0, 0, 1, 0,
       1, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0,
       0, 1, 1, 0, 1, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0,
       0, 1, 0, 1, 1, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0,
       0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0,
       0, 1, 0, 1, 1, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0,
       0, 0, 0, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 1,
       0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1,
       1, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 1, 1,
       1, 1, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 0,
       0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 0, 0,
       0, 1, 1, 1, 0, 1, 1, 1, 0, 1])

```

```

[ ] #tarkistetaan, että bike_target:in tietotyyppi on numpy
    #tekoälyn koulutus ei onnistu, jos tietotyyppi ei ole numpy, esim. panda
    type(bike_target)

```

```

numpy.ndarray

```

```
[ ] #tarkistetaan, että bike_target:illa on oikea määrä rivejä (1000)
    #jolloin bike_target:in sisältö vastaa itse bike_data:n (ja new)
    bike_target.shape
```

↔ (1000,)

```
[ ] #jaetaan koulutusjoukko bike_data_newest dataa käyttäen 75% ja 25% suhteella
    X_train, X_test, y_train, y_test = train_test_split(bike_newest, bike_target, train_size=0.75, test_size=0.25)
    X_train.shape, X_test.shape, y_train.shape, y_test.shape
```

↔ ((750, 26), (250, 26), (750,), (250,))

```
# Koulutetaan tpot luokittelija, hyväksikäyttäen yllä tehtyä datan jakoa
# Huom: jos haluaa paremman luokittelijan, voi lisätä max_time_min parametria, esim = 10.
tpot = TPOTClassifier(verbosity=2, max_time_mins=2)
tpot.fit(X_train, y_train)
print(tpot.score(X_test, y_test))
```



Optimization Progress: 76%  153/200 [01:55<00:42, 1.11pipeline/s]

2.02 minutes have elapsed. TPOT will close down.

TPOT closed during evaluation in one generation.

WARNING: TPOT may not provide a good pipeline if TPOT is stopped/interrupted in a early generation.

TPOT closed prematurely. Will use the current best pipeline.

Best pipeline: KNeighborsClassifier(SelectPercentile(input_matrix, percentile=52), n_neighbors=51, p=1, weights=distance)
0.672

```
[ ] #exportataan tpot pipeline, mikäli sitä tarvitaan myöhemmin,
    #esimerkiksi jos halutaan tarkastella tpot luokittelijan tarkkuutta
    tpot.export('tpot_bike_pipeline.py')
```

```
[ ] bike.to_csv('out.csv', sep=',')
```

```
[ ] import numpy as np
import pandas as pd
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures

# NOTE: Make sure that the class is labeled 'target' in the data file
tpot_data = pd.read_csv('out.csv', sep=',', dtype=np.float64)
features = tpot_data.drop('target', axis=1).values
training_features, testing_features, training_target, testing_target = \
    train_test_split(features, tpot_data['target'].values, random_state=None)
```

```
▶ import numpy as np
import pandas as pd
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures

# NOTE: Make sure that the class is labeled 'target' in the data file
tpot_data = pd.read_csv('out.csv', sep=',', dtype=np.float64)
features = tpot_data.drop('target', axis=1).values
training_features, testing_features, training_target, testing_target = \
    train_test_split(features, tpot_data['target'].values, random_state=None)

# Average CV score on the training set was:0.6919713172437294
exported_pipeline = make_pipeline(
    PolynomialFeatures(degree=2, include_bias=False, interaction_only=False),
    GradientBoostingClassifier(learning_rate=0.1, max_depth=10, max_features=0.25,
                               min_samples_leaf=17, min_samples_split=8, n_estimators=100, subsample=1.0)
)

exported_pipeline.fit(training_features, training_target)
results = exported_pipeline.predict(testing_features)
```

```
[ ] #Tarkistetaan miten tarkka meidän malli on
from sklearn.metrics import accuracy_score
accuracy_score(testing_target, results)
```

↔ 0.736



```
#ladataan data jota ennustetaan

# Alla oleva koodi ei toimi
#bike_sub = pd.read_csv('Bike_data_potentialiset.csv')
#bike_sub.head()

# Uusi koodi:
# Huom: Bike_data.csv on ladattu minun google driveen.

bike_sub_file_path = '/content/drive/My Drive/Bike_data.csv'

bike_sub = pd.read_csv(bike_sub_file_path)

bike.head()
```



	ID	Income	Children	Cars	Age	target	Marital Status = Married	Marital Status = Single	Gender = Female	Gender = Male	...	Occ
0	-1.396956	-0.519278	-0.558393	-1.291005	-0.192891	0	0.924355	-0.924355	1.017656	-1.017656	...	
1	0.774406	-0.841012	0.671548	-0.401883	-0.104813	0	0.924355	-0.924355	-0.981668	0.981668	...	
2	-1.082594	0.767657	1.901490	0.487239	1.392518	0	0.924355	-0.924355	-0.981668	0.981668	...	
3	0.825647	0.445923	-1.173364	-0.401883	-0.280970	1	-1.080754	1.080754	-0.981668	0.981668	...	
4	1.053050	-0.841012	-1.173364	-1.291005	-0.721361	1	-1.080754	1.080754	-0.981668	0.981668	...	

5 rows × 28 columns

```
[ ] #Poistetaan ennustettavasta datasta tiedot ID ja target
bike_sub_new = bike_sub.drop(['ID','target'], axis=1)
```

```
[ ] #Tarkistetaan bike_sub_new:n tietotyyppi, joka on pandas
type(bike_sub_new)
```



```
pandas.core.frame.DataFrame
def __init__(data=None, index: Axes | None=None, columns: Axes | None=None, dtype: Dtype |
None=None, copy: bool | None=None) -> None
```

Two-dimensional, size-mutable, potentially heterogeneous tabular data.

Data structure also contains labeled axes (rows and columns). Arithmetic operations align on both row and column labels. Can be thought of as a dict-like container for Series objects. The primary pandas data structure.


```
[ ] #Muunnetaan bike_sub_new taulun tietotyyppi numpy:ksi
    #Toimii myös pandas tietotyyppillä
    bike_sub_newest = bike_sub_new.values
```

```
[ ] #tarkistetaan, että bike_sub_new on numpy
    type(bike_sub_new)
```



```
pandas.core.frame.DataFrame
def __init__(data=None, index: Axes | None=None, columns: Axes | None=None, dtype: Dtype |
None=None, copy: bool | None=None) -> None
```

Two-dimensional, size-mutable, potentially heterogeneous tabular data.

Data structure also contains labeled axes (rows and columns).
Arithmetic operations align on both row and column labels. Can be
thought of as a dict-like container for Series objects. The primary
pandas data structure.

```
[ ] #tarkistetaan, että bike_sub_new:issä on oikea määrä rivejä ja sarakkeita
    bike_sub_new.shape
```

(1000, 26)

```
[ ] #Tehdaan ennustus tpotin aiemmin kouluttamallamme luokittelijalla
    submission = tpot.predict(bike_sub_new)
```

/usr/local/lib/python3.10/dist-packages/sklearn/base.py:486: UserWarning: X has feature names, but SelectPercentile was fitted without feature names
warnings.warn(

```
[ ] #Katsotaan, miltä meidän ennustusdata näyttää
    #(1 tarkoittaa, että ostaa pyörän, 0 että ei osta)
    submission[:]
```

[illegible]

```
[ ] # Tehdään-csv tiedosto ennusteista
final = pd.DataFrame({'ID': bike_sub['ID'], 'target': submission})
final.to_csv('submission24.csv', index = False)
```

```
#tarkistetaan osa 15kpl/78kpl ennustuksista. Tässä näkyy vielä ostajan ID targetin vieressä.
final.head(15)
```



	ID	target
0	-1.396956	0
1	0.774406	0
2	-1.082594	1
3	0.825647	1
4	1.053050	1
5	-1.207890	0
6	1.497570	1
7	-0.112578	1
8	0.409364	0
9	-0.128287	1
10	0.412731	1
11	-1.359368	0
12	-1.595560	0
13	1.001809	1
14	0.668746	1

Yhteenveto:

Tehtiin tekoälyluokittelija, joka kykenee ennustamaan saamansa datan perusteella ennustamaan ostaako asiakas polkupyörän. Luokittelijan tarkkuus 73.6 %. Tarkemman luokittelijan saa, jos antaa luokittelijan optimointiin enemmän aikaa: tässä työssä annettiin aikaa 2 minuuttia, tätä aikaa voi nostaa paljon korkeammaksi.

Luokittelijaa käytettiin datajoukkoon (Bike_data_potentialiset.csv), jossa oli vain asiakkaan tiedot, mutta ei tietoja siitä, ostaako asiakas polkupyörän. Luokittelijaa käyttämällä teimme ennusteita siitä, ostavatko asiakkaat tällä listalla polkupyörän, ja loimme CSV-tiedoston, jossa näkyy asiakkaat (ID), joiden on ennustettu ostavan polkupyörän (target arvo on 1).

Tällaista tietoa voi käyttää esimerkiksi asiakastarjousten kohdentamiseen niille asiakkaille, jotka on tunnistettu todennäköisiksi asiakkaita.